



UNIVERSITÀ DI PISA

DIPARTIMENTO DI INFORMATICA

Report Assignment 2

Student:

Stefano Pea

ANNO ACCADEMICO 2024/2025

1 Execution Instructions

In order to test the programs, simply run the `script_sbatch.sh` script using the `sbatch` command inside the `spmcluster` machine. For example:

```
sbatch script_sbatch.sh
```

This command will run `test_script.sh`, that will compile both the sequential and the parallel versions of the algorithm, and then it will start a series of tests changing scheduling protocol, chunk size and number of threads.

2 Implementation choices

For this assignment two different algorithms have been developed in order to solve the maximum Collatz length problem: a sequential one and a parallelized one, using both static and dynamic scheduling policies.

2.1 `collatz_seq`

The file `collatz_seq` contains the sequential implementation of the algorithm. Its structure is very simple: for each range passed as an argument to the program, it executes the function `max_collatz_in_range` and at the end it will have computed the maximum length of this range. The time starts from the moment that the main thread starts to compute the first max length and ends after the last computation.

2.2 `collatz_par`

The file `collatz_par` instead contains the parallel version of the algorithm. According to the assignment, two different kind of task scheduling have been implemented and tested:

- A static policy using a block-cycle approach.
- A dynamic policy using a Thread Pool with a Shared Queue.

2.2.1 Static Scheduling

The static scheduling policy uses a **block-cycling** approach to assign tasks to each worker. To calculate the values that should be computed, a thread only needs:

- Its id (`t`),
- The start and end of the range (`start` and `end`),
- The chunk size (`chunk_size`).

With these values, the thread can independently calculate the numbers it needs to process without requiring additional synchronization or communication. The local maximum that each one computes is then inserted in a shared vector called `local_max_values`. Even though this structure is not explicitly thread-safe (as all threads can access it without locks), the safety of this approach is guaranteed by the facts that each thread gets assigned an index in the array and will only modify the value in this position, without accessing regions belonging to other threads. After all the workers complete their tasks, the `local_max_values` vector will contain all the local maxima. The main thread can then calculate the final result by finding the maximum value in this vector.

2.2.2 Dynamic Scheduling

The dynamic scheduling policy uses a Thread Pool to manage tasks. In this approach:

- A Thread Pool is initialized with a fixed number of threads, specified by the user.
- The range of numbers is divided into chunks, and each chunk is treated as an independent task, which are dynamically pulled by the workers from the shared queue, ensuring that no thread remains idle after completing a task.

Each chunk is processed by a thread, which computes the maximum sequence length across all numbers in that region. This partial results are retrieved thanks to a `std::future`. The final result is then calculated by comparing each value encountered in this way.

2.2.3 Theoretical Comparison

Table 2.1 summarizes the main advantages and disadvantages that each policy introduces.

Block-Cyclic Static	
Advantages:	Low overhead, predictable
Disadvantages:	Load imbalance
Dynamic Scheduling	
Advantages:	Balanced
Disadvantages:	High overhead, sync delays

Table 2.1: Comparison of Advantages and Disadvantages for the scheduling policies

For the static policy, having fixed chunks of work assigned before execution greatly reduces overhead, and makes the behavior highly predictable, since we know

in advance which computations each thread will handle. The problem comes from the fact that dividing the ranges in chunks, even in the case of the block cyclic variant of our implementation, offers no guarantee that the amount of work will be evenly distributed among threads.

The dynamic one solves this problem by using a shared queue from which each thread extract tasks the moment that it finishes with the previous one. In order to do this, it needs to use a shared queue introducing synchronization overhead.

3 Work-Span Model

The Work-Span model evaluates parallel algorithms based on two metrics:

- **Work** (T_1): total number of operations done by the program, either parallel or sequential.
- **Span** (T_∞): The total number of operations on the critical path of the computation

In our case, we can define both of them in this way:

- The total **Work** is given by:

$$T_1 = \sum_{\text{range} \in \text{Ranges}} \sum_{\text{value} \in \text{range}} \text{sequence_length}(\text{value})$$

This represents the total number of operations performed over all values in all input ranges.

- The **Span** is defined as:

$$T_\infty = \max_{\text{value} \in \text{range}} \text{sequence_length}(\text{value})$$

That is, the longest sequence length encountered among all values across all ranges — effectively, the length of the critical path in the computation.

Using these definitions, we can apply **Brent's Theorem**, which provides a bound on the execution time T_p of a parallel algorithm on p processors:

$$T_p \leq \frac{T_1}{p} + T_\infty$$

It implies that adding more processors beyond a certain point yields diminishing returns due to the span.

From Brent's Theorem, we can also derive the upper bound for parallel speedup:

$$S_p \leq \min(p, \frac{T_1}{T_\infty})$$

with p = number of processors.

This inequality expresses the fact that speedup is limited both by the number of available processors and by the structure of the computation itself.

To illustrate this with a practical example, consider the following test configuration:

- Ranges = $[10^{18}-1, 000000000001 \times 10^{18}]$, $[100000-110000]$, $[1000000-2000000]$
- $t = 32$
- $c = 1024$
- scheduling = dynamic

From the computation, we obtain:

$$T_1 = 4243270092$$

$$T_\infty = 1126$$

Substituting these values into the speedup formula:

$$S_p \leq \min(32, \frac{4243270092}{1126}) = \min(32, 3768445) = 32$$

Thus, the theoretical maximum speedup in this case is limited by the number of processors, not by the algorithm's structure, and it could have been expected given the extremely parallelizable nature of our problem. In the following performance analysis, we will check how well our implementation behaves.

4 Performance Analysis

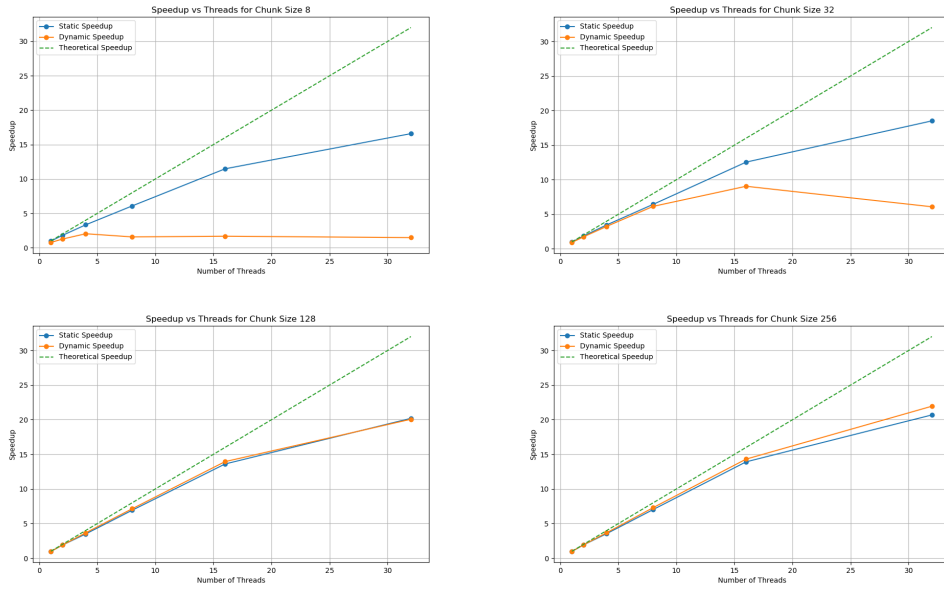
To check whether our parallel implementations align with the theoretical expectations, we conducted a series of experiments. These tests were made using the same interval ranges for the sequential and the parallel programs, in order to obtain the values of the strong scalability of our program. all tests were performed using the following fixed input ranges, the same as the example above:

1000000000000000000 – 1000000000010000000,
 100000 – 110000,
 1000000 – 2000000

These ranges were chosen because empirical observations suggest that the higher the starting value, the longer the corresponding Collatz sequence tends to be. Additionally, the large size of this range helps mitigate the overhead of the parallel implementations by increasing the overall computational load.

The graphs in Figure 4 show the observed speedup for both static and dynamic scheduling across different numbers of threads (1, 2, 4, 8, 16 and 32) and chunk sizes (8, 16, 32, 64, 128, 256, 512, 1024 and 2048).

Multiple considerations can be done on the results of our tests:

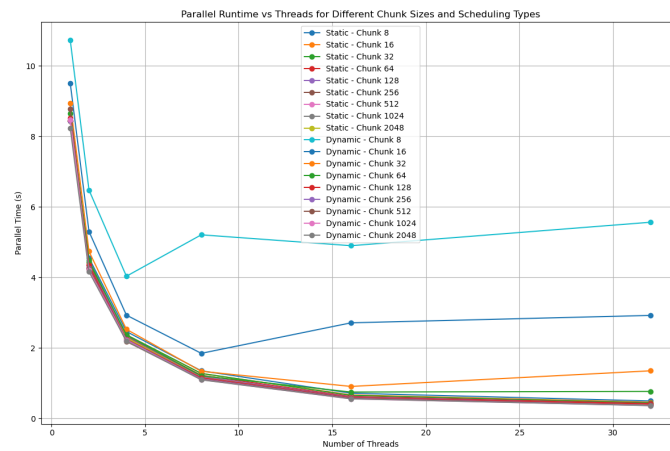


- When the chunk size is small, the dynamic implementation suffers from high overhead due to synchronized accesses to the shared Queue. As a result, the static version significantly outperforms the dynamic one in these cases.
- As the chunk size increases, the dynamic implementation starts to become more efficient. At chunk size of 128, it surpasses the static version in terms of speedup. This could be explained by the fact that the amount of work for each task is dominating the time to retrieve the task itself.
- Both strategies started to depart from the theoretical speedup as the number of threads increases. In the dynamic case, this degradation is likely caused by a greater number of accesses to the queue. In the static case, the slowdown is not due to shared memory access (each thread writes in its own section of the vector) but rather due to creation and deletion of threads for each range, which introduces some synchronization overhead. Other than that, Amdahl's Law states that the speedup of a parallel program is limited by the sequential portion of the code. In our implementation the computation of the global maximum (`std::max_element` or reduction of `local_max_values`) is sequential, and this step could be a significant factor contributing to the observed discrepancy in speedup, as it becomes a bottleneck when the number of threads increases.

Concluding Remarks

Despite the observed limitations, both scheduling strategies performed decently well. As seen in Figure 4, they show similar execution times and speedup under optimal conditions. However, the dynamic strategy becomes preferable for larger chunk sizes, when the cost of task scheduling is offset by a sufficiently large computation

per chunk.



The implementations could still be improved by implementing a better Thread Pool for the dynamic scheduling policy and by reusing threads for the static one.